into one branch (for example, 0 – 2 – 6 – 14) before returning to explore other branches. These differences make BFS more suitable for finding the shortest path in unweighted graphs, while DFS is often employed for tasks like exploring all possible configurations or detecting cycles in graphs.

Now, let us perform BFS and DFS on a cycle to better understand the differences between the two algorithms. The program '*cycle19.sagews*' shown below generates a cycle called *Cycle_19*.

```
C = graphs.CycleGraph(15)
C.show( )
```

Upon executing the program 'cycle*19.sagews*' using CoCalc, we obtain the cycle *Cycle_19*, as shown in Figure 5.

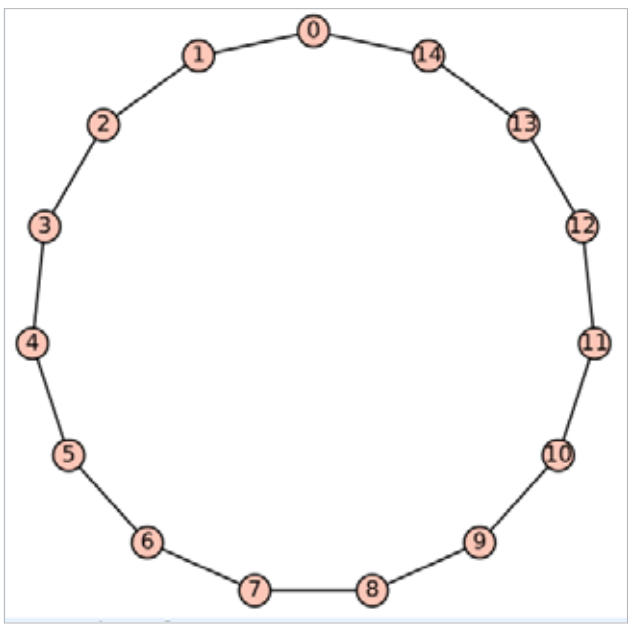

Figure 5: The cycle *Cycle_19*

Now, let us perform BFS on the cycle *Cycle_19*. To do this, add the following lines of code at the end of the '*cycle19.sagews*' program. The BFS will begin from vertex 14.

```
bfs_order = list(C.breadth_first_search(14))
print("BFS Order:", bfs_order)
```

Upon executing the modified program, the following additional output will be displayed on the screen.

```
BFS Order: [14, 0, 13, 1, 12, 2, 11, 3, 10, 4, 9, 5, 8, 6, 7]
```

Now, let us perform DFS on the cycle *Cycle_19*. To do this, add the following lines of code at the end of the '*cycle19.sagews*' program. The DFS will begin from vertex 14.

```
dfs_order = list(C.depth_first_search(14))
print("DFS Order:", dfs_order)
```

Upon executing the modified program, the following additional output will be displayed on the screen.

```
DFS Order: [14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Now, let us compare the BFS and DFS traversal sequences. BFS begins at the initial vertex (14 in this case) and explores all its immediate neighbours before progressing to the neighbours of those vertices. This approach leads to a traversal pattern that alternates between moving left and right along the cycle, covering the vertices in a more evenly distributed manner. On the other hand, DFS also starts at vertex 14 but dives as deeply as possible along one path before backtracking. In this scenario, it traverses the entire cycle in a single direction, visiting each vertex in sequence until it reaches the last vertex (0 in this case).

Before we conclude our discussion of graphs in this SageMath series, we need to cover one more important type of graph: planar graphs. A planar graph is a graph that can be drawn on a flat surface, like a piece of paper, in such a way that its edges do not cross each other. In other words, you can arrange the vertices (points) and edges (lines connecting the points) of the graph so that no two edges intersect except at their endpoints (vertices). If it is impossible to draw a graph without edges crossing, then that graph is non-planar. For example, a triangle is a planar graph because you can easily draw it without any edges crossing. Now, let us check whether *Cycle_19* is planar or not. To do this, add the following line of code at the end of the program '*cycle19.sagews*'.

```
C.is_planar( )
```

Upon executing the modified program, the message '*True*' is displayed, verifying that *Cycle_19* is indeed planar, as can be easily confirmed from Figure 5. Please also verify that *Wgraph_17* and *Tree_18* are planar graphs.

Now, let us explore an important aspect of planar graphs: a planar graph can be drawn in a non-planar way. This means that even though the graph can be represented without any crossing edges, it might still be drawn in a way that causes some edges to intersect. Take, for example, the complete graph K_4, which consists of four vertices, with every pair of vertices connected by an edge. K_4 is a planar graph because it can be drawn without any edges crossing. The code below displays the complete graph K_4 and also checks whether it is planar.